

# Handoff #3: G1, the cascade, and Airline

Everything since handoff #2 (commit 3130cb1): Kev-4B/9B run for real on Gilbreth, a rigorous statistical audit, a live confidence-gated cascade decider, and a first (partial) run on the real Airline domain — including the funding wall we hit and what it taught us.

## TL;DR

- G1 ran clean. Kev-4B and Kev-9B replayed on the mock pilot with real, validated GPU energy — no fabricated numbers, one live SLURM bug found and fixed (- - exclusive was silently grabbing both GPUs on a shared node).
- A paired-bootstrap audit found the mock pilot's headline claim was overstated. "Kev-4B  $\approx$  Kev-9B" survived the test (not significant, CI touches 0) — but that null result turned out to be a mock-only artifact.
- Built a live confidence-gated cascade (Kev-4B GPT-OSS-20B, threshold 0.9): +8.3pts over Kev-4B alone, not yet significant at  $n=65$ , real API errors handled and resumed cleanly.
- Airline broke the mock-only story. On 662 real Airline states, Kev-4B vs Kev-9B reverses and becomes significant (-12.2pts,  $p<0.05$ ) — the opposite of mock. Both models' accuracy collapses on Airline (Kev-4B 61.5%28.9%, Kev-9B 63.1%41.1%, both significant).
- Ran out of real money mid-collection. OpenRouter account hit \$0 (of \$5) after two real bugs — a 20 req/min new-account cap, then an uncapped 65536-token credit pre-check — both diagnosed and fixed, but Airline coverage stopped at 70% (105/150 episodes; tasks 38-49 have zero episodes). Jev, GPT-OSS-20B and the cascade on Airline are blocked until credits are added.

## Your first 5 minutes

```
git pull # gets the cascade code, the airline states/results, this PDF
less results/replay/mock-pilot/post_experiment_analysis.json # the significance tests
less results/replay/airline/report/summary.md # Kev-4B/9B on real Airline data
less results/replay/compare-mock-pilot-vs-airline/comparison.md # mock vs airline, side by side
```

## 1 · G1: Kev-4B and Kev-9B, for real, on Gilbreth

Ran the two Slurm replay jobs the handoff #2 asked for, against the same 65-state mock pilot. Both completed with validated GPU energy (no !! GPU energy not measured anywhere in either log) and clean data — 325/325 calls each, 0 errors, prefix cache confirmed off.

| Decider           | Accuracy [95% CI]    | p50 / p95 latency | J / decision (gross / net)     |
|-------------------|----------------------|-------------------|--------------------------------|
| Jev (API)         | 0.772 [0.668, 0.868] | 0.374 / 0.510 s   | n/a (hosted)                   |
| GPT-OSS-20B (API) | 0.852 [0.778, 0.917] | 0.873 / 1.737 s   | n/a (hosted)                   |
| Kev-0.8B (laptop) | 0.354 [0.246, 0.477] | 0.131 / 0.182 s   | not measured (invalid counter) |
| Kev-4B (A100)     | 0.615 [0.492, 0.738] | 0.086 / 0.101 s   | 18.33 / 11.95                  |
| Kev-9B (A100)     | 0.631 [0.508, 0.754] | 0.093 / 0.120 s   | 24.44 / 17.93                  |

### A real SLURM bug, found live

--exclusive in kev\_replay.slurm reserves every GPU on the node, not just the one requested — a 2-GPU node handed both cards to a job that asked for 1, burning 2 of the account's 3-GPU group cap and starving a labmate's queued job. Fixed: dropped --exclusive, kept --gres=gpu:1 (SLURM's cgroup isolation already gives sole ownership of the one GPU, which is all the energy counter needs). Kev-9B's first attempt also OOM'd at --mem=32G (needs ~48G in fp32-before-cast); fixed by requesting 96G.

Full validation pass on the run: row counts match expected exactly (65 states × 5 repeats), no warmup-call leakage into the logged data, no cache contamination, no silently-skipped states.

## 2 • The rigor pass: what actually holds up

Built `post_experiment_analysis.py` — paired cluster-bootstrap significance tests, data-integrity checks, per-state cross-tabs, post-hoc calibration, and a cascade simulation — all computed from the raw `calls-*.jsonl`, not eyeballed off the summary tables.

### Significance, not just overlap

| Comparison (mock, n=65 states) | Diff      | 95% CI        | Significant? |
|--------------------------------|-----------|---------------|--------------|
| Kev-4B vs Kev-9B               | -1.5 pts  | [-4.6, +0.0]  | No           |
| Jev vs Kev-9B                  | +14.2 pts | [-1.5, +29.5] | No (!)       |
| Jev vs GPT-OSS-20B             | -8.0 pts  | [-14.8, -1.8] | Yes          |

The eye-catching “Jev beats Kev-9B by 14 points” framing from the pilot report does not survive a paired test at n=65 — direction is right, significance isn’t there yet. Needs ~10× more states to nail down at ±5pt precision.

### Calibration: temperature scaling actually works for Kev

| Decider     | ECE before | ECE after (held-out) | Fitted T |
|-------------|------------|----------------------|----------|
| Jev         | 0.109      | 0.148 (worse)        | 0.7-1.6  |
| GPT-OSS-20B | 0.111      | 0.057                | 1.8-2.2  |
| Kev-4B      | 0.307      | 0.059                | 5.5-6.0  |
| Kev-9B      | 0.321      | 0.080                | 5.0-5.2  |

2-fold, state-level split (fit on one half, score on the other) — not graded on its own training data. Kev’s confidence is dramatically overconfident and a single scalar fixes most of it; Jev is already well calibrated and scaling makes it slightly worse (its reliability curve isn’t monotonic).

### Error analysis: the same failure mode, at every Kev size

Kev-4B beats Kev-9B: 0 states. Kev-9B beats Kev-4B: 1 state. Scaling 4B9B unlocks essentially nothing state-by-state on mock — the clearest possible confirmation that the null result above is real, not just noise. Jev beats both Kev models on 19 states, clustered in tasks where Kev redundantly re-acts (`update_task_status`) after the task is already done — the exact same “called tool instead of replying” pattern flagged for Kev-0.8B in handoff #2, still present at 4B and 9B, just less often.

### 3 · The cascade: built live, not just simulated

Kev-4B decides first; below a confidence threshold, escalates to GPT-OSS-20B. Reuses the existing Decider interface (CascadeDecider works with any two deciders, not just this pair) — 3 new unit tests, all 29 project tests pass.

#### One deliberate design call

No live GPU energy metering on the cascade run itself. Wrapping the whole `decide()` call (Kev stage + a possibly multi-second network escalation) in the same NVML window used for pure-Kev runs would count the GPU's idle power during that wait as "decision energy." Kev-4B's own validated 18.33 J/decision is reused post-hoc instead.

#### Threshold sweep (simulated from replayed calls) live run at $t=0.9$

Pareto-optimal thresholds: 0.0, 0.5, 0.6, 0.9, 0.95, 1.0 (full escalation). Two different "optimal" answers depending on what you weight: the algorithmic pick (min distance-to-utopia) is  $t=0.6$  (conservative, +1.8pts, barely escalates); the best accuracy-for-cost trade — what actually got run live — is  $t=0.9$ .

| Decider (mock, $n=65$ )   | Accuracy | 95% CI       | Median lat. | p95 lat.  |
|---------------------------|----------|--------------|-------------|-----------|
| Kev-4B                    | 61.5%    | [49.2, 73.8] | 86.0 ms     | 100.6 ms  |
| Kev-9B                    | 63.1%    | [50.8, 75.4] | 92.8 ms     | 120.0 ms  |
| Cascade ( $t=0.9$ , live) | 69.8%    | [59.1, 79.7] | 249.2 ms    | 2004.8 ms |
| Jev                       | 77.2%    | [66.8, 86.8] | 373.8 ms    | 510.2 ms  |
| GPT-OSS-20B               | 85.2%    | [77.8, 91.7] | 872.8 ms    | 1736.6 ms |

#### Honest headline: the live gain isn't proven yet

The live cascade's +8.3pt gain over Kev-4B alone does not clear significance at  $n=65$  (diff +8.3pts, CI [-1.5, +18.5]). Only cascade-vs-GPT-OSS-20B is significant (cascade is still worse, as expected — diff -15.4pts, CI [-25.8, -4.9]). Two real API errors hit during the live run (OpenRouter occasionally returns no tool call for GPT-OSS-20B) — same pre-existing flakiness seen before, resumed cleanly to 325/325 clean rows.

## 4 · Airline: the funding saga

Domain verified clean first (50/50 tasks, 14 tools, policy + eval fields all load). Model chosen:

openrouter/openai/gpt-5.6. Dry run (5 tasks × 3 trials) succeeded 15/15, \$0.30 total — but that clean result turned out to be not representative of the full run.

### *Three real bugs, each one diagnosed live from the actual error, not guessed*

1. OpenRouter 20 req/min new-account cap.

First attempt (unthrottled, 4 workers): all 15 episodes failed. tau2's own 3-retry default can't outlast a hard per-minute quota under concurrent load. Fixed: a rate limiter monkey-patching the exact bound name tau2 uses internally (tau2.utils.llm\_utils.completion — patching litellm.completion itself would NOT have worked, tau2 already copied the reference at import time).

2. Sustained load still degraded — 32% of the full 150-episode run failed (48 episodes), concentrated as a solid failed tail (tasks 38-49, 12 tasks with ZERO episodes, not scattered noise).

3. Root cause: max\_tokens was uncapped at 65536 (the model's own ceiling). OpenRouter pre-authorizes credits against that worst case on every call, before any tokens are generated or billed — “you requested up to 65536 tokens, but can only afford 32405.” Fixed with a max\_tokens cap (4096) — the same monkeypatch mechanism as the rate limiter.

The fix worked, but the account ran out of real money before the retry finished. Balance check via OpenRouter's own /credits API: \$5.00 total, \$5.00 used, \$0 remaining. Confirmed, not inferred. Every further OpenRouter call — including Jev, GPT-OSS-20B, and the cascade's escalation stage — is blocked until credits are added.

### *What's actually usable right now*

| Metric                                            | Value            |
|---------------------------------------------------|------------------|
| States collected (merged across all attempts)     | 1,065            |
| Episodes covered                                  | 105 / 150 (70%)  |
| Episodes with reward=1 (usable for replay labels) | 77               |
| Usable states for replay                          | 689              |
| Tasks with ZERO episodes                          | 38-49 (12 tasks) |

## 5 · Airline results: does mock generalize? No.

Kev-4B and Kev-9B replayed on all 689 usable Airline states (5 repeats each) — real, validated GPU energy again. 135/3445 calls per decider hit Kev's genuine 8192-token context limit on the longest Airline conversations (correctly excluded from accuracy, not silently dropped, and identical across both models since it's state-length-driven, not model-specific).

### Finding 1: the mock “no benefit from scaling” conclusion reverses

| Comparison (Airline, n=662 states) | Diff      | 95% CI        | Significant? |
|------------------------------------|-----------|---------------|--------------|
| Kev-4B vs Kev-9B                   | -12.2 pts | [-15.6, -8.9] | YES          |

Kev-9B wins 108 states to Kev-4B's 27 (4:1). On mock this same comparison was not significant (CI touched 0). Airline — harder domain, far more data — reveals scaling really does help; mock was too small/easy to see it.

### Finding 2: absolute accuracy collapses on the harder domain

| Decider | Mock acc. | Airline acc. | Diff      | 95% CI         | Sig? |
|---------|-----------|--------------|-----------|----------------|------|
| Kev-4B  | 61.5%     | 28.9%        | +32.7 pts | [+20.0, +44.8] | YES  |
| Kev-9B  | 63.1%     | 41.1%        | +22.0 pts | [+9.2, +34.1]  | YES  |

Independent-sample bootstrap (different state universes, not paired). Mock's 5-tool toy domain was not predictive of real tool-selection difficulty at 14-tool scale with much longer contexts (3,099 vs 430 median input tokens).

### Finding 3: a whole new failure mode appears

“Repeated tool call” goes from nearly negligible on mock (1 instance total) to a major error category on Airline. Longer, multi-turn Airline conversations (up to 14 turns) create many more chances for Kev to redundantly re-invoke a tool it already called — a failure mode mock's short conversations structurally couldn't expose.

Bottom line: any paper claim built only on the mock pilot needs an explicit Airline (or harder-domain) caveat — especially “no benefit from scaling Kev,” which Airline actively contradicts. The energy/latency story (Kev-4B faster & lower-energy than Kev-9B) likely still holds directionally, but now trades against a real, significant accuracy cost mock hid. Unresolved: whether Jev and the cascade show the same generalization gap — needs OpenRouter funding to test.

## 6 · What's next

### Blocked on you

Add OpenRouter credits to unblock: the remaining 45 Airline episodes (tasks 38–49), plus Jev / GPT-OSS-20B / cascade replay on Airline — all currently impossible at \$0 balance.

### Ready to run the moment funding lands

- Complete Airline state collection for tasks 38–49 (script already fixed: rate-limited + max\_tokens-capped).
- Replay Jev, GPT-OSS-20B, and the cascade ( $t=0.9$ ) on the 689 (soon ~1,000+) Airline states.
- Extend the significance/comparison scripts (already airline-ready, crash-fixed for missing deciders) to the full 5-decider + cascade picture.
- Test whether the cascade's generalization gap matches Kev-4B/9B's — the paper's core mechanism, still unverified past mock.

### New files this handoff

```
src/dwg/decisions.py # + CascadeDecider, Decision.escalated
scripts/kev_cascade_replay.slurm, scripts/make_cascade_figure.py
scripts/post_experiment_analysis.py # significance, calibration, cascade sim
scripts/make_paper_figures.py, scripts/montage_figures.py, scripts/compare_domains.py
scripts/collect_decision_states.py # + rate_limit_tau2_llm_calls, cap_max_tokens
results/replay/{mock-pilot,airline}/paper_figures/, post_experiment_analysis.json
results/replay/compare-mock-pilot-vs-airline/
results/states/airline.jsonl # 1,065 states, 105/150 episodes
```

### Gotchas we hit this round

- SLURM --exclusive on a shared multi-GPU node silently grabs every GPU, not just the one requested — check AllocTRES, not just your own --gres request.
- Gilbreth now requires an explicit --mem on every job (policy change since handoff #2's scripts were written).
- OpenRouter's credit pre-check is against max\_tokens requested, not actual usage — always cap it explicitly for cost-sensitive collection runs.
- tau2 binds `from litellm import completion` at import time — patch `tau2.utils.llm_utils.completion`, not `litellm.completion`, or your monkeypatch silently does nothing.